MCMP117 · Week 3 · Course Notes

Fundamentals of Operating Systems

Operating-System Services, System Calls & Processes

What does the OS do for a program — and what exactly is a running program?

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do after this week

1. List the services an operating system provides to users and programs, and the services it needs for its own efficient operation.
2. Distinguish an API function, a library wrapper and a system call, and trace one `write()` from a C program into the kernel and back.
3. Explain the system-call table, the three ways of passing parameters, and the six categories of system calls.
4. Explain the difference between a program and a process and draw the memory layout of a process (text, data, bss, heap, stack).
5. Draw the five-state process diagram, say what triggers each transition, and list the contents of a process control block (PCB).
6. Describe a context switch and compute its overhead; explain `fork()`, `exec()` and `wait()`, zombies and orphans, and count the processes created by a loop of `fork()` calls.

Contents

1 Operating-System Services 1

2 System Calls in Detail 3

3 The Process Concept 8

4 Process States 11

5 The Process Control Block 13

6 Context Switch 14

7 Process Creation and Termination 15

8 Summary 19

9 Glossary 20

10 Try It Yourself 20

11 Review Questions 21

12 Looking Ahead: Week 4 22

Where we left off

In Week 2 we met **dual-mode operation** and watched a first **system call** with `strace`: `printf()` ended up as `write(1, …)`, and a minimal system call cost about 116 ns, roughly 76 times a function call. This week we ask which services sit behind the system-call door (Section 1) and how a request travels through it (Section 2); then we turn to the **process** (Sections 3–7).

1 Operating-System Services

An operating system provides an environment in which programs run. The specific services differ from one OS to another, but the common ones fall into two groups: services that help the *user* (and the user's programs), and services that keep the *system itself* running efficiently and safely when many users and programs share it.

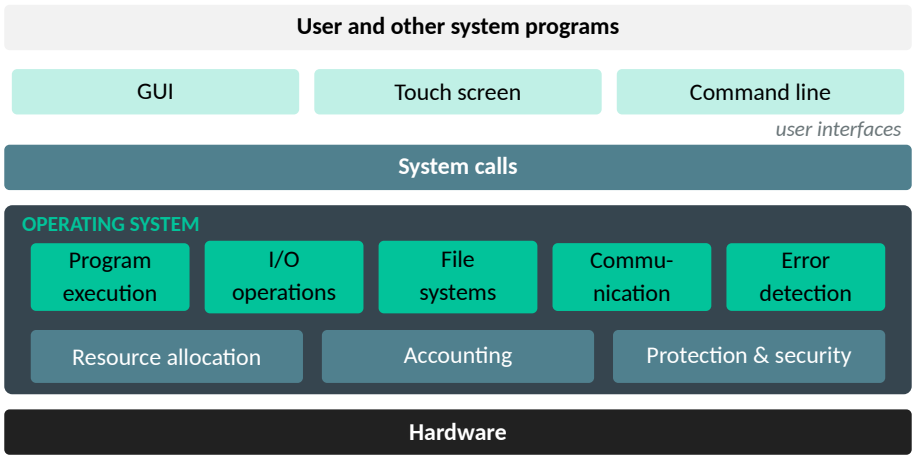


Figure 1: A view of operating-system services (after Silberschatz et al., Fig. 2.1). Mint: services for the user; slate: services for the system itself.

1.1 Services that help the user

- **User interface.** Almost every OS has one. A **command-line interface** (CLI) reads typed commands; a **graphical user interface** (GUI) uses windows, icons and a pointer; phones and tablets use a *touch-*

screen interface; and a *batch* interface reads commands from a file.

- **Program execution.** The OS loads a program into memory, runs it, and ends it, either normally or abnormally (with an error code).
- **I/O operations.** A running program may need a file or a device. Because of dual mode, programs cannot drive devices themselves; the OS performs the I/O for them.
- **File-system manipulation.** Programs create, delete, read, write and search files and directories, and manage their permissions.
- **Communication.** Processes exchange information, either on the same computer or between computers over a network. This is done with **shared memory** (two processes read and write the same memory area) or with **message passing** (the OS moves packets of data between processes, e.g. pipes and sockets).
- **Error detection.** Errors can happen in the CPU and memory, in devices (a disk read fails, a network link goes down, the printer runs out of paper) and in user programs (division by zero, an access to a forbidden address). For each type of error the OS must take the right action: retry, report, or terminate the program.

1.2 Services for the system itself

- **Resource allocation.** When several users or jobs run at the same time, each must get its share of the CPU, memory, files and devices. CPU schedulers (Weeks 4–5) and memory allocators (Weeks 6–7) do this work. On Linux, `ulimit -a` shows the per-process limits the OS enforces.
- **Accounting and logging.** The OS keeps track of which users and programs use how much of which resource, for billing, statistics and tuning. `ps`, `top` and the system log (`journalctl`) on Linux, or the Event Viewer on Windows, show this information.
- **Protection and security.** *Protection* means controlling the access of processes to resources: one process must not read or overwrite another's memory. *Security* means defending the system against attacks from outside and inside: users must authenticate (usually with a password), and attempts to break in are logged.

A chain is only as strong as its weakest link: all of the protection hardware of Week 2 is worthless if an attacker can guess the administrator's password.

1.3 User interfaces: command line and GUI

The program that reads and executes typed commands is called the **command interpreter** or **shell**. On Linux and macOS the common shells are `bash` and `zsh`; on Windows, `cmd.exe` and PowerShell. Two points are important:

1. The shell is *not* part of the kernel. It is an ordinary system program running in user mode, using exactly the same system calls as any other program.
2. Most commands are themselves separate programs. When you type `ls`, the shell finds the program file (`/usr/bin/ls`) and asks the OS to run it as a new process (Section 7 shows how).

A command line is fast for experienced users and easy to automate with scripts; a GUI is easier to learn.

From Week 9 onward we work in the Linux shell.

2 System Calls in Detail

2.1 API, library wrapper and system call

Programmers almost never execute a system call directly. Between a line such as `printf("x = %d\n", x);` and the hardware there are several layers.

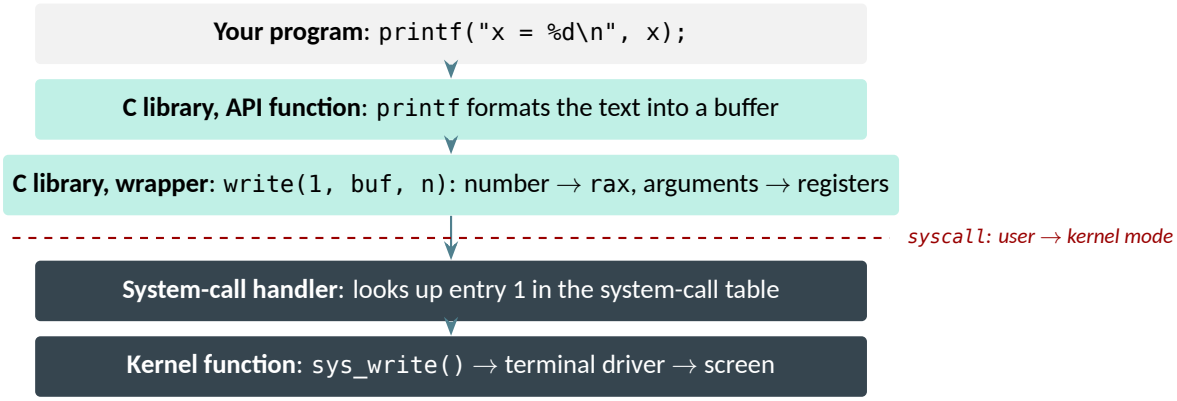


Figure 2: From `printf()` to the screen. Above the dashed line: user mode; below it: kernel mode.

Three words are easily confused:

- An **API** (application programming interface) is a documented set of functions that a programmer can call, together with their parameters and return values. Examples: the *POSIX* API (Linux, macOS, the BSDs), the *Windows API*, and the *Java API*.
- A **wrapper** is a tiny library function, usually with the same name as the system call (`write`, `read`, `fork`), whose only job is to put the arguments where the kernel expects them and execute the trap instruction.
- A **system call** is the actual, controlled entry into the kernel.

Not every API function makes a system call. `strlen()` and `sqrt()` never do: they compute their result in user mode. `printf()` makes a system call only when its buffer is flushed, so ten calls to `printf()` may produce a single `write`. The C library on Linux is usually *glibc*; on Windows, the Windows API functions in `kernel32.dll` call the native interface in `ntdll.dll`, which executes the real kernel calls (e.g. `CreateProcess()` eventually calls `NtCreateUserProcess`).

2.2 Why program against an API?

The main reason is **portability**. A program written for the POSIX API compiles on any POSIX system, whereas system-call *numbers* and details differ between operating systems and even between CPU architectures of the same OS: `write` is system call number 1 on 64-bit x86 Linux, 4 on 32-bit x86 Linux and 64 on 64-bit ARM Linux, while the API name `write()` is the same on all three. A second reason is **convenience**: API functions are easier to use and often add useful work, such as the formatting and buffering of `printf()`.

2.3 The system-call number and the system-call table

Every system call is identified by a small integer, the **system-call number**. The kernel keeps a **system-call table**: an array, indexed by this number, of pointers to the kernel functions that do the work.

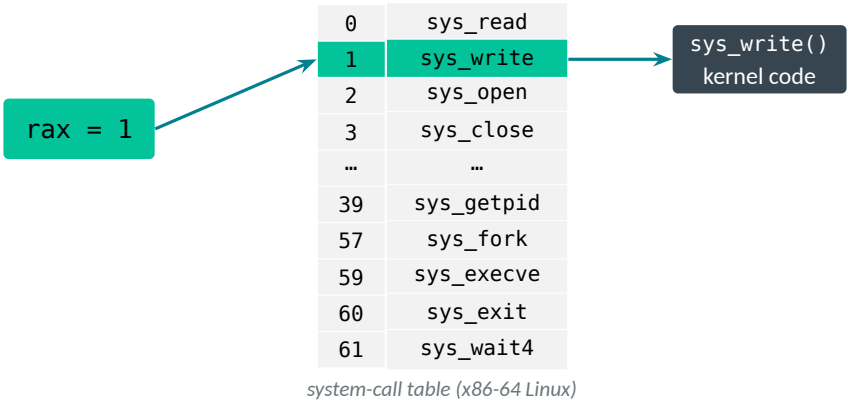


Figure 3: The number in `rax` selects an entry of the system-call table. The numbers are the real x86-64 Linux numbers.

Why a table, and not simply a jump to the address of `sys_write`? Because a user program must *never* be allowed to jump to an arbitrary kernel address: it could skip the permission checks at the start of a kernel function. The trap instruction therefore always enters the kernel at *one* fixed entry point. The kernel checks that the number is valid (an invalid number returns the error `ENOSYS`) and only then calls the function from the table. Compare this with the interrupt vector of Week 2: both are tables of handler addresses indexed by a number. On our test system (Linux headers for kernel 6.8, x86-64) about 370 system calls are defined, numbered from 0 to 461.

2.4 The trap, step by step

Here is what happens when a program calls `write(1, msg, 26)` on 64-bit x86 Linux:

1. **Load registers** (user mode). The `write()` wrapper in the C library puts the system-call number 1 in register `rax` and the three parameters in `rdi`, `rsi` and `rdx`.
2. **Trap.** The wrapper executes the `syscall` instruction. The CPU switches the mode bit to kernel mode, saves the return address and flags, and jumps to the kernel's single system-call entry point.
3. **Save and check** (kernel mode). The kernel saves the user's registers and checks that 1 is a valid system-call number.
4. **Dispatch.** The kernel calls `sys_write()` through the table. `sys_write()` checks that file descriptor 1 is open and that the buffer address really belongs to the calling process, then passes the bytes to the terminal driver.
5. **Result.** The return value, the number of bytes written (26) or a negative error code, is placed in `rax`.
6. **Return.** The `sysret` instruction restores the user state and switches back to user mode. The wrapper returns 26 to the program.

The check in step 4 matters: without it, a program could ask the kernel to "write" bytes from a kernel address to the screen and so read kernel secrets. A system call is a request, and the kernel trusts nothing

in it.

2.5 Three ways to pass parameters

A system call usually needs more information than just its number: which file, which buffer, how many bytes. There are three general methods to pass these parameters to the OS.

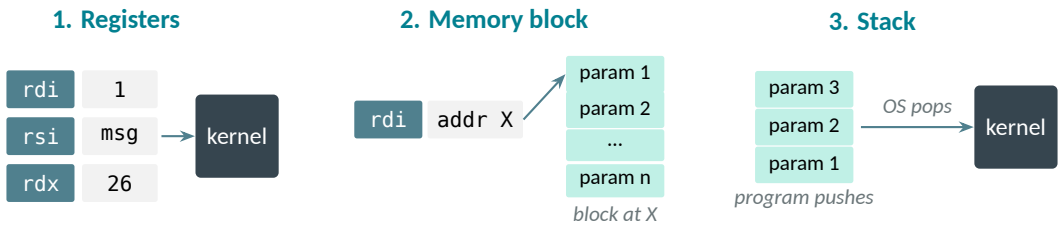


Figure 4: Three ways of passing parameters to a system call.

1. **In registers.** The fastest method: no memory access is needed. Its limit is the number of available registers; x86-64 Linux allows six parameters.
2. **In a block (table) in memory.** The program stores the parameters in a block of memory and passes only the block's *address* in a register. Any number of parameters is possible. Linux uses this for system calls with many options; for example, `clone3()` receives a pointer to a `struct clone_args`.
3. **On the stack.** The program *pushes* the parameters onto its stack; the OS *pops* (reads) them from there. This was used, for example, by 32-bit FreeBSD on x86.

Methods 2 and 3 have a hidden cost: the kernel must copy the parameters from user memory into kernel memory and check that the addresses really belong to the calling process. Linux combines methods 1 and 2: up to six parameters in registers, and anything larger (a buffer, a structure) passed as an address in a register. Note that even in method 1 the 26 bytes of the message stay in user memory; only their address travels in a register.

2.6 Concrete example: the x86-64 Linux convention

Register	Role in a system call	For `write(1, msg, 26)`
`rax`	system-call number (in), return value (out)	1 in, 26 out (bytes written)
`rdi`	1st parameter	1 (file descriptor: standard output)
`rsi`	2nd parameter	address of `msg`
`rdx`	3rd parameter	26 (number of bytes)
`r10`, `r8`, `r9`	4th, 5th, 6th parameter	not used by `write`
`rcx`, `r11`	overwritten by the `syscall` instruction	the CPU saves the return address and the flags here

Ordinary C function calls on x86-64 pass the fourth argument in `rcx`; the system-call convention uses `r10` instead, because the `syscall` instruction itself overwrites `rcx`. A return value between -4095 and -1

signals an error: the wrapper stores the positive error number in the global variable `errno` and returns -1 to the program. You do not need to memorise the registers; the important idea is that parameters travel in registers.

2.7 Example 1: the same message, three levels deep

The programs in this section are in the `code/` folder next to these notes, with a `Makefile` (type `make`). They are written in C for Linux; as in Week 2, you are not expected to write C, only to read the comments and understand what happens. The first program prints three messages: through `printf()`, through the `write()` wrapper, and by executing the `syscall` instruction by hand with a few lines of inline assembly that do exactly what the wrapper does.

```c
/* raw_write.c -- one message, three levels: library, wrapper, raw trap.
 * Shows how a system call is identified by a NUMBER and how its
 * parameters travel in CPU registers (x86-64 Linux only).
 * Compile:  gcc -Wall -O2 raw_write.c -o raw_write
 * Trace:    strace ./raw_write
 */
#include <stdio.h>    /* printf: C library (user mode) */
#include <unistd.h>   /* write : thin libc wrapper     */

int main(void)
{
    printf("1) printf  -> libc -> write\n");     /* level 1 */
    fflush(stdout);

    /* level 2 */
    if (write(1, "2) write() -> libc wrapper\n", 27) != 27)
        return 1;

    /* level 3: do what the wrapper does, by hand.
     *   rax = 1   system-call number of write (x86-64)
     *   rdi = 1   1st parameter: file descriptor (stdout)
     *   rsi = msg 2nd parameter: address of the bytes
     *   rdx = 26  3rd parameter: number of bytes
     * The 'syscall' instruction traps into the kernel; the
     * kernel's return value comes back in rax.          */
    static const char msg[] = "3) syscall instruction!!!\n";
    long ret;
    __asm__ volatile ("syscall"
                      : "=a"(ret)                /* out: rax */
                      : "a"(1L), "D"(1L), "S"(msg), "d"(26L)
                      : "rcx", "r11", "memory"); /* clobbered */
    printf("kernel returned %ld (bytes written)\n", ret);
    return 0;
}
```

```
$ make raw_write && strace ./raw_write
...
write(1, "1) printf  -> libc -> write\n", 28) = 28
write(1, "2) write() -> libc wrapper\n", 27) = 27
write(1, "3) syscall instruction!!!\n", 26) = 26
write(1, "kernel returned 26 (bytes writte"..., 35) = 35
exit_group(0)                           = ?
```

Two observations. First, `strace` cannot tell the three levels apart: to the kernel, all three are the same system call number 1 with three parameters. Second, the kernel returned 26 in `rax`, exactly the number of bytes we asked it to write. The inline assembly is for demonstration only and works only on x86-64 Linux; on ARM processors the trap instruction is `svc` and different registers are used.

2.8 Six categories of system calls

System calls can be grouped into six categories. The table gives typical examples; the Windows column lists Windows API functions, each of which calls one or more native system calls.

Category	Typical requests	Linux (POSIX)	Windows API
Process control	create, end, load, wait	`fork()`, `execve()`, `exit()`, `wait4()`	`CreateProcess()`, `ExitProcess()`, `WaitForSingleObject()`
File management	create, open, read, write, close	`open()`, `read()`, `write()`, `close()`	`CreateFile()`, `ReadFile()`, `WriteFile()`, `CloseHandle()`
Device management	request, read, write, configure	`ioctl()`, `read()`, `write()`	`SetConsoleMode()`, `ReadConsole()`, `WriteConsole()`
Information maintenance	time, date, IDs, system data	`getpid()`, `uname()`, `clock_gettime()`	`GetCurrentProcessId()`, `GetSystemTime()`
Communication	pipes, shared memory, network	`pipe()`, `mmap()`, `socket()`	`CreatePipe()`, `MapViewOfFile()`
Protection	permissions, ownership	`chmod()`, `chown()`, `umask()`	`SetFileSecurity()`, `SetSecurityDescriptorGroup()`

Notice that `read()` and `write()` appear under both file and device management. In UNIX a device looks like a special file (for example `/dev/tty` for the terminal), so the same system calls serve both. This is the extended-machine view of Week 2 at work: one abstraction, many kinds of hardware.

How many system calls does copying a file take?

A program that copies one file to another needs, in the classic textbook description:

1. Ask for the two file names (I/O system calls, or read them from the command line).
2. Open the input file; if it does not exist, print an error message and exit (file management, process control).
3. Create the output file.
4. Loop: read a block from the input, write it to the output, until `read()` returns 0 (end of file).
5. Close both files, report success, and exit.

With a buffer of B bytes and a file of S bytes, the loop alone makes $\lceil S/B \rceil$ reads that return data, one final read that returns 0, and $\lceil S/B \rceil$ writes. For $S = 1$ MiB $= 2^{20}$ bytes and $B = 4096 = 2^{12}$ bytes that is $2 \cdot 2^8 + 1 = 513$ system calls.

The real GNU `cp` (version 9.4) on our test machine behaves differently. `strace -c cp in.bin out.bin` for a 1 MiB file counted 102 system calls in total, the largest groups being `mmap` (26), `close` (13), `openat` (11), `fstat` (10) and `read` (9) — and just 2 calls of `copy_file_range`. Only 2 of the 102 calls move data: `copy_file_range` asks the kernel to copy the whole file *inside the kernel*, without

passing the bytes through the program's memory. Most of the remaining calls load the program and its shared libraries (`mmap`, `openat`, `read` of library headers). Moving work into the kernel to save system calls is a recurring theme in OS design.

2.9 System programs

System programs (also called system utilities) ship with the OS and provide a convenient environment for program development and execution. They run in user mode and make system calls on your behalf. Most users see the operating system through them, not through system calls. The main categories are: **file management** (`ls`, `cp`, `rm`, File Explorer); **status information** (`date`, `df`, `ps`, `top`, Task Manager, the system logs); **file modification** (editors such as `nano` and Notepad, search tools such as `grep`); **programming-language support** (compilers, debuggers, interpreters: `gcc`, `gdb`, `python`); **program loading and execution** (loaders and linkers); **communications** (`ssh`, web browsers, e-mail); and **background services**, programs started at boot that run all the time, called **daemons** in UNIX and *services* in Windows (`sshd`, `cron`, the print spooler).

Many of these programs will be used hands-on in Weeks 9–14.

3 The Process Concept

3.1 Program versus process

A **program** is a *passive* entity: a file on disk containing a list of instructions, such as `/usr/bin/bash`. A **process** is an *active* entity: a program in execution. Besides the program code, a process has

- a *current activity*: the value of the **program counter** (the address of the next instruction) and of the other CPU registers;
- its own *memory*: a stack, a heap and global data (Section 3.2);
- resources given to it by the OS: open files, network connections, and so on.

A program becomes a process when it is loaded into memory, for example when you double-click an icon or type its name in a shell. One program can be the basis of many processes: if you open two terminal windows, two `bash` processes run from the same program file, each with its own memory and its own progress. Older books use the word *job* (from batch systems) for the same idea.

Analogy: a recipe is a program; cooking it is a process. Two cooks can follow the same recipe at the same time, each with their own ingredients and their own position in the recipe.

3.2 A process in memory

The memory of a process is divided into regions (also called segments or sections):

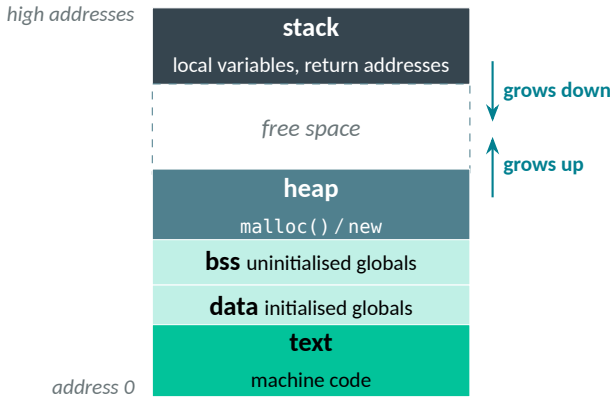


Figure 5: Layout of a process in memory.

- **Text:** the compiled machine code of the program. Read-only, so that a bug cannot overwrite the code; several processes running the same program can share it.
- **Data:** global (and `static`) variables that have an initial value, e.g. `int initialised = 42;`. The values are stored in the executable file.
- **BSS:** global variables *without* an initial value, e.g. `int uninitialised;`. The OS fills this region with zeros when the program is loaded, so the executable file needs to store only its size. (The name is historical: "block started by symbol", from an old assembler.)
- **Heap:** memory requested while the program runs, with `malloc()` in C or `new` in C++ and Java. It lives until it is explicitly freed.
- **Stack:** one *frame* per active function call, holding its local variables, parameters and return address. A frame is removed when its function returns.

Text, data and bss have a fixed size once the program is loaded; the heap and the stack grow and shrink while it runs, towards each other into the free space between them. A stack that grows too far, for example through infinite recursion, causes a *stack overflow* and the program is terminated.

3.3 Example 2: printing the address of each region

```c
/* mem_layout.c -- where do the parts of a process live in memory?
 * Prints one address from each region of the process image.
 * Compile:  gcc -Wall -O2 mem_layout.c -o mem_layout
 * Run:      ./mem_layout   (run it twice: the addresses move,
 *                           because Linux randomises them: ASLR)
 */
#include <stdio.h>
#include <stdlib.h>

int initialised = 42;   /* global with a value    -> data */
int uninitialised;      /* global without a value -> bss  */

int main(void)
{
    int local = 7;                  /* local variable -> stack */
    int *dynamic = malloc(sizeof *dynamic);     /* malloc -> heap */

    printf("text  main()        %p\n", (void *)main);
```

```
    printf("data  initialised   %p\n", (void *)&initialised);
    printf("bss   uninitialised %p\n", (void *)&uninitialised);
    printf("heap  *dynamic      %p\n", (void *)dynamic);
    printf("stack local         %p\n", (void *)&local);

    free(dynamic);
    return 0;
}
```

```
$ ./mem_layout
text  main()        0x555d9ba160c0
data  initialised   0x555d9ba19010
bss   uninitialised 0x555d9ba19018
heap  *dynamic      0x555db649d2a0
stack local         0x7ffe2b4f6ab4
```

The order of the addresses matches the figure: text $<$ data $<$ bss $<$ heap, with the stack far above. In the first run the data variable lies `0x555d9ba19010` $-$ `0x555d9ba160c0` $=$ `0x2f50` $= 12\,112$ bytes above `main`, and the bss variable only 8 bytes above the data variable: they sit next to each other. The gap between heap and stack is `0x7ffe2b4f6ab4` $-$ `0x555db649d2a0` $\approx 4.7 \times 10^{13}$ bytes, about 43 TiB, far more than the RAM of any PC. This is possible because these are *virtual* addresses; only the parts actually used are backed by physical memory (Week 7).

Run the program again and every address is different: in our second run, for example, `main` was at `0x55a4811220c0` and `local` at `0x7ffec440c1f4`. Linux places the regions at random positions each time a program starts, a technique called **address space layout randomisation** (ASLR). An attacker who finds a bug in a program then cannot know in advance where its code and data are.

> **ASLR**
>
> ASLR was introduced by the PaX security project for Linux in 2001 and entered the standard Linux kernel in 2005; Windows adopted it with Windows Vista (2007). Today it is standard on desktop and mobile systems.

4 Process States

As a process executes, it changes **state**. Each process is in one of five states:

- **New:** the process is being created.
- **Ready:** the process is waiting to be assigned a CPU.
- **Running:** its instructions are being executed.
- **Waiting** (also *blocked*): the process is waiting for some event, such as the completion of an I/O operation or the end of a child process.
- **Terminated:** the process has finished execution.

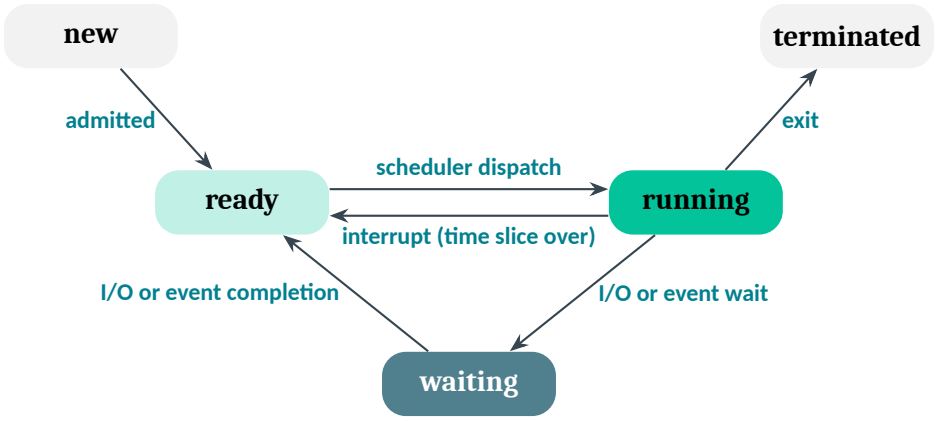


Figure 6: The five-state process model.

Only one process can be *running* on a CPU core at any instant; many processes may be *ready* or *waiting*. The OS keeps the ready processes in a **ready queue** and the waiting processes in **wait queues** (one per device or event); each queue is a linked list of process control blocks (Section 5).

4.1 What triggers each transition

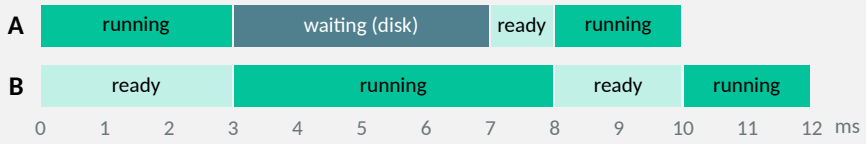
Transition	Triggered by	Everyday example
new → ready	the OS has created the PCB and loaded the program	you type `./mem_layout` and press Enter
ready → running	the scheduler *dispatches* the process	a CPU core becomes free and the scheduler picks it
running → ready	an interrupt, usually the timer (time slice used up)	a long calculation is paused so that others can run
running → waiting	a system call that must wait (I/O, child, sleep)	`read()` from the disk; `wait()` for a child
waiting → ready	the awaited event happens, often signalled by an interrupt	the disk interrupt says the data has arrived
running → terminated	`exit()` or a fatal error	`main()` returns; the program divides by zero

Two remarks. First, a process can never go from *waiting* directly to *running*: when its event happens it joins the ready queue and must wait until the scheduler dispatches it. Which ready process runs next is the subject of *CPU scheduling* (Weeks 4–5). Second, notice who causes the transitions: a process leaves the running state either by making a system call (Section 2) or because an interrupt (Week 2) takes the CPU away. The OS is interrupt-driven.

On Linux, the `ps` command shows the state in its STAT column: `R` for running *or* ready (Linux does not distinguish them there), `S` for sleeping (waiting, interruptible), `D` for waiting uninterruptibly (usually for disk I/O), `T` for stopped and `Z` for zombie (Section 7.4).

Two processes, one CPU

Process A computes for 3 ms, reads from the disk (4 ms), then computes for 2 ms more. Process B needs 7 ms of CPU time. The OS gives each process a time slice of at most 5 ms. Both are ready at $t = 0$, and A is dispatched first.



- $t = 3$: A calls `read()` and becomes *waiting*; B is dispatched.
- $t = 7$: the disk interrupt moves A to *ready*; B keeps the CPU.
- $t = 8$: the timer interrupt ends B's 5 ms slice; B becomes ready and A is dispatched.
- $t = 10$: A exits; B runs its remaining 2 ms and exits at $t = 12$.

The CPU work needed is $3 + 2 = 5$ ms for A and 7 ms for B, 12 ms in total. Run one after the other, A would take $3 + 4 + 2 = 9$ ms (with the CPU idle for 4 ms) and B another 7 ms: 16 ms in total, with the CPU busy $12/16 = 75\%$ of the time. Interleaved as above, both finish after 12 ms and the CPU is busy $12/12 = 100\%$ of the time. This is the multiprogramming idea of Week 1, now described with process states.

5 The Process Control Block

Each process is represented in the OS by a **process control block** (PCB), also called a *task control block*. It is a data structure in kernel memory that holds everything the OS needs to know about the process.

process state
process ID (PID), parent PID
program counter
CPU registers
CPU-scheduling information
memory-management information
accounting information
I/O status: list of open files, devices

Figure 7: Typical contents of a process control block.

- **Process state:** new, ready, running, waiting or terminated.
- **Identification:** the process ID (PID), the parent's PID, the owner (user ID).
- **Program counter:** the address of the next instruction to execute.
- **CPU registers:** their values must be saved when the process stops running, so that it can continue correctly later.
- **CPU-scheduling information:** priority, pointers into the scheduling queues.

- **Memory-management information:** where the regions of the process are, e.g. base and limit registers or page tables (Weeks 6–7).
- **Accounting information:** CPU time used, time limits, number of context switches.
- **I/O status information:** the list of open files and allocated devices.

When a process is not running, its PCB *is* the process as far as the CPU is concerned: it contains everything needed to resume it. In Linux the PCB is a C structure called `struct task_struct`, defined in the kernel source file `include/linux/sched.h`; it has hundreds of fields.

5.1 Peeking at a PCB: `/proc/<pid>/status`

Linux makes parts of every PCB visible as text files in the `/proc` directory. `/proc` is not a real directory on disk: the kernel generates the text on the fly when you read it. For a `sleep` process on our test machine (the `grep` pattern is broken over two lines here; type it on one line):

```
$ sleep 30 &
$ grep -E "^(Name|State|Pid|PPid|Uid|VmSize|VmRSS|Threads|voluntary)" \
       /proc/$!/status
Name:   sleep
State:  S (sleeping)
Pid:    27271
PPid:   27265
Uid:    0       0       0       0
VmSize:     2724 kB
VmRSS:      1696 kB
Threads:        1
voluntary_ctxt_switches:        2
nonvoluntary_ctxt_switches:     0
```

`$!` is the PID of the most recent background command. The fields map directly onto the PCB: `State` is the process state (`sleep` waits for a timer, so it is `S`); `Pid` and `PPid` identify the process and its parent; `Uid` is the owner (here user 0, the administrator `root`); `VmSize` (virtual memory size) and `VmRSS` (the part actually in RAM) are memory-management information; the two context-switch counters are accounting information. *Voluntary* switches happen when the process gives up the CPU by waiting; *non-voluntary* switches happen when the OS takes the CPU away. The `Threads` line is next week's topic.

6 Context Switch

When an interrupt or a system call causes the OS to take the CPU from one process and give it to another, the OS performs a **context switch**. The *context* of a process is represented in its PCB: the program counter, the CPU registers, the stack pointer, the process state and the memory-management information (which page table is active). A context switch consists of two steps:

1. **State save:** store the context of the current process in its PCB.
2. **State restore:** load the context of the next process from its PCB.

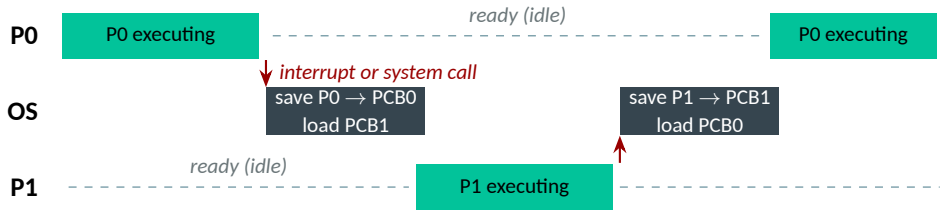


Figure 8: A context switch from P0 to P1 and back (after Silberschatz et al., Fig. 3.6, drawn with time running left to right).

During the switch (the dark boxes) *no process does useful work*: the context switch is pure **overhead**. Its direct cost on modern hardware is a few microseconds; it depends on the CPU speed, the number of registers to copy and the memory-management work. There is also a hidden indirect cost: after a switch, the CPU caches and the TLB (Week 6) are full of the previous process's data, so the new process runs more slowly for a while.

How much does context switching cost?

Suppose every process runs for a time slice q before the OS switches, and each switch costs s. Of every period $q + s$, only q is useful, so the fraction of CPU time lost is

$$\text{overhead} = \frac{s}{q + s}.$$

Take $s = 5\ \mu$s (an illustrative value):

Time slice q	Overhead $s/(q+s)$	Useful work
100 ms	0.005%	99.995%
10 ms	0.05%	99.95%
1 ms	0.50%	99.50%
0.1 ms	4.76%	95.24%
0.01 ms	33.3%	66.7%
0.005 ms ($q = s$)	50%	50%

For example, with $q = 1$ ms $= 1000\ \mu$s: $5/(1000 + 5) = 0.004975 \approx 0.50\%$. Equivalently, 1000 switches per second cost $1000 \times 5\ \mu\text{s} = 5$ ms of every second, i.e. 0.5%.
The lesson: short time slices make the system feel responsive but waste CPU time on switching; long time slices waste little but make interactive programs feel sluggish. Choosing q is a central question of Round-Robin scheduling in Week 5.

7 Process Creation and Termination

7.1 fork(), exec() and wait()

A process can create new processes. The creator is the **parent process**; the new processes are its **child processes**. In UNIX and Linux, three system calls do the work:

- `fork()` creates a new process that is an almost exact *copy* of the caller: same program, same memory contents, same open files. The only difference is the return value: `fork()` returns 0 in the child and the child's PID in the parent (or -1 if no process could be created). It is called once and returns twice.
- `exec()` (in Linux the system call `execve()`, with library variants such as `execlp()`) *replaces* the program running in the calling process by a new program. The PID stays the same; the old code, data, heap and stack are discarded. On success `exec()` never returns.
- `wait()` (or `waitpid()`) puts the parent into the *waiting* state until a child terminates, and then collects the child's exit status.

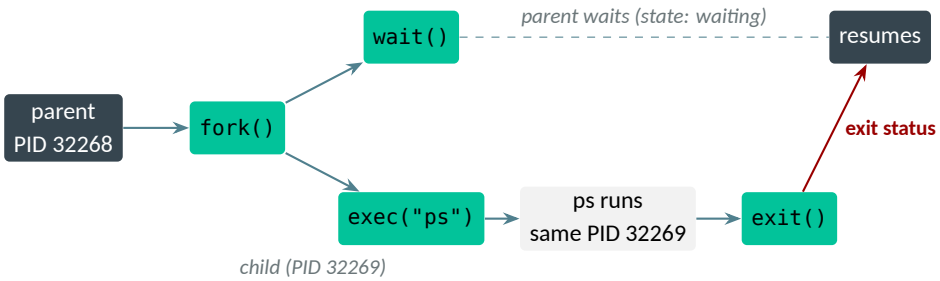


Figure 9: Process creation with `fork()`, `exec()` and `wait()`. PIDs from the real run below.

This is exactly what a shell does for every command you type: it forks a copy of itself, the copy (the child) execs the command, and the shell waits for it to finish before printing the next prompt. Why two separate steps? Between `fork()` and `exec()`, the child can prepare its environment, for example redirect its output into a file. We will use this in Week 12 (pipes and redirection). Windows combines both steps in one call, `CreateProcess()`, which takes the name of the program to run as a parameter.

7.2 Example 3: fork, exec and wait in C

The listing shows the heart of `code/fork_exec_wait.c`; the full file adds the `#include` lines and an error check for a failed `fork()` (which returns -1).

```c
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);            /* empty the buffer BEFORE fork */

    pid_t pid = fork();        /* one process calls, two return */

    if (pid == 0) {            /* ---- child: fork returned 0 */
        printf("child : PID %d, my parent is %d\n",
               getpid(), getppid());
        fflush(stdout);
        char list[32];         /* "parent,child" for ps -p */
        snprintf(list, sizeof list, "%d,%d", getppid(), getpid());
        execlp("ps", "ps", "-o", "pid,ppid,stat,comm",
               "-p", list, NULL);   /* replaces this program */
        exit(127);             /* reached only if exec failed */
    }
    int status;                /* ---- parent */
    waitpid(pid, &status, 0);  /* sleep until child ends */
```

```
    printf("parent: child %d finished, exit status %d\n",
           pid, WEXITSTATUS(status));
}
/* full program, with error checks: code/fork_exec_wait.c */
```

```
$ ./fork_exec_wait
parent: my PID is 32268
child : PID 32269, my parent is 32268
  PID  PPID STAT COMMAND
32268 32251 S    fork_exec_wait
32269 32268 R    ps
parent: child 32269 finished, exit status 0
```

The same `if` test gives two different answers: the child sees `pid == 0`, the parent sees the child's PID. The `ps` output, printed by the child after `exec()`, lists PID 32269 as `ps`: the process kept its PID but now runs a different program. Its state is R (it is running: it is the process doing the listing), while the parent is S, waiting in `waitpid()`.

Under `strace -f` the three calls appear as `clone(…) = 6411`, `execve("/usr/bin/ps", …) = 0` and `wait4(6411, …) = 6411` (from another run): the C library implements `fork()` with the more general Linux system call `clone()`.

> **Why the `fflush(stdout)` before `fork()`?**
>
> `printf()` collects text in a buffer inside the process's memory. `fork()` copies that memory, including any text not yet written. Remove the first `fflush(stdout)`, run `./fork_exec_wait > out.txt` (output to a file is fully buffered), and look at `out.txt`: the line "parent: my PID is …" appears twice, once printed by each process.

7.3 The process tree and process identifiers

Every process has a unique integer **process identifier** (PID) and knows the PID of its parent (PPID). Because every process except the first is created by another, the processes form a **process tree**. On Linux the root of the user-space tree is PID 1, called `systemd` (or `init` on older and smaller systems), which the kernel starts at the end of booting; kernel threads hang below a separate root, `kthreadd` (PID 2).

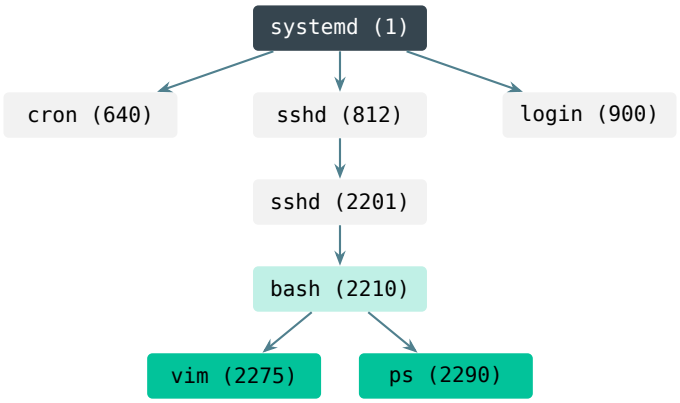


Figure 10: Part of a typical Linux process tree (PIDs illustrative). A user logged in over SSH runs `vim` and `ps` from a `bash` shell.

The command `pstree -p` draws the real tree. On our test machine:

```
$ bash -c 'sleep 5 & sleep 5 & pstree -p $$'
bash(6297)-+-pstree(6300)
           |-sleep(6298)
           `-sleep(6299)
```

`$$` is the PID of the shell itself. The new `bash` created three children, all forked from it. PIDs are assigned in increasing order and reused after a process has ended; the largest PID is configured in `/proc/sys/kernel/pid_max` (32768 on our test machine).

7.4 Process termination, zombies and orphans

A process terminates when it calls `exit()` (returning from `main()` does this implicitly) or when the OS kills it after a fatal error. The OS then frees its memory and closes its files. But the PCB entry with the PID and the *exit status* must survive until the parent collects it with `wait()`.

- A **zombie** is a process that has terminated but whose parent has not yet called `wait()`. It uses no memory or CPU, but it occupies an entry in the process table and keeps its PID. A parent that creates many children and never waits for them can fill the process table, so that no new processes can be created.
- An **orphan** is a process whose parent terminated first. Linux *re-parents* orphans to PID 1 (or to a designated "subreaper" process), which calls `wait()` regularly, so orphans never remain zombies for long. Some systems instead terminate all children when a parent ends (*cascading termination*).

A zombie is easy to produce with a shell trick: `sh -c 'sleep 1 & exec sleep 4'` starts `sleep 1` in the background and then *replaces* the shell by `sleep 4`, a program that never calls `wait()`. Two seconds later, on our test machine:

```
$ ps -o pid,ppid,stat,cmd -p 6385 --ppid 6385
  PID  PPID STAT CMD
 6385  6384 S    sleep 4
 6387  6385 Z    [sleep] <defunct>
```

The finished `sleep 1` is shown in state Z and marked `<defunct>`. When `sleep 4` ends two seconds later, the zombie becomes an orphan, is adopted by its new parent and is reaped.

7.5 Example 4: how many processes after n calls to fork()?

The program `code/fork_count.c` reads n from the command line and then executes

```c
for (int i = 0; i < n; i++)
    fork();                  /* parent AND child continue */
printf("PID %6d  parent %6d\n", getpid(), getppid());
while (wait(NULL) > 0)       /* reap all children: no zombies */
    ;
```

Every *existing* process executes each subsequent `fork()`. Before the first fork there is 1 process; after it, 2; after the second, 4; after the n-th, 2^n. Of these, $2^n - 1$ were created by `fork()` (all except the original),

and the `printf` runs 2^n times. A real run with $n = 3$:

```
$ ./fork_count 3
PID   1939  parent   1937
PID   1938  parent   1937
PID   1940  parent   1937
PID   1941  parent   1938
PID   1942  parent   1939
PID   1943  parent   1938
PID   1937  parent   1903
PID   1944  parent   1941
```

Eight lines, from eight different processes. The parent column reveals the tree: the original process 1937 created 1938, 1939 and 1940 in forks 1, 2 and 3; 1938 (born in fork 1) itself executed forks 2 and 3, creating 1941 and 1943; and so on. The order of the lines changes from run to run, because the scheduler decides which process runs first.

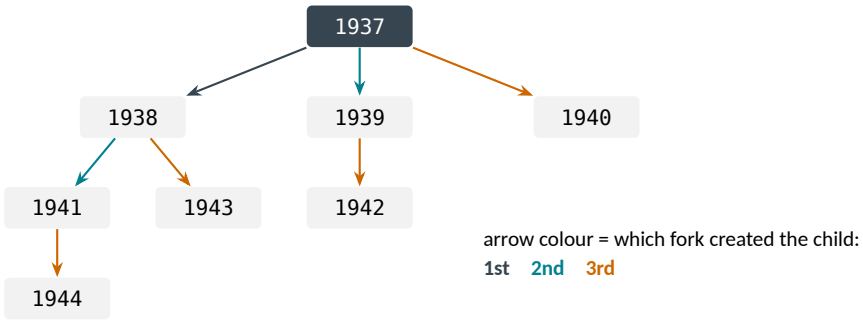


Figure 11: The process tree of `./fork_count 3`, with the real PIDs.

The formula can be verified by counting output lines:

```
$ for n in 0 1 2 3 4 5 6 7 8 9 10; do ./fork_count $n | wc -l; done | tr '\n' ' '
1 2 4 8 16 32 64 128 256 512 1024
```

The program refuses $n > 10$ for a good reason: a loop that forks without limit (a "fork bomb") fills the process table within seconds. Linux limits the number of processes per user (`ulimit -u`), an example of the resource-allocation service of Section 1.

8 Summary

Week 3 in seven sentences

1. The OS offers services for the user (user interface, program execution, I/O, file systems, communication, error detection) and for itself (resource allocation, accounting, protection and security).
2. Programs reach these services through system calls, normally via an API and a thin library wrapper; the shell and the other system programs are ordinary user-mode programs.
3. Each system call has a number; the trap instruction enters the kernel at one fixed point, and the kernel uses the number as an index into the system-call table.

4. Parameters are passed in registers, in a memory block whose address is in a register, or on the stack; x86-64 Linux uses `rax` for the number and up to six registers for parameters.
5. A process is a program in execution, with text, data, bss, heap and stack regions, and is described by its process control block.
6. A process moves between the states new, ready, running, waiting and terminated; system calls and interrupts cause the transitions.
7. A context switch saves one PCB and loads another and is pure overhead, $s/(q+s)$; processes are created with `fork()`, `exec()` and `wait()`, and n forks in a loop give 2^n processes.

9 Glossary

API Application programming interface: a documented set of functions that programs can call, e.g. POSIX or the Windows API.

ASLR Address space layout randomisation: placing the regions of a process at random addresses at every start.

BSS The region holding global variables without an initial value; filled with zeros at load time.

Context switch Saving the state of the running process in its PCB and loading the state of another process from its PCB.

Daemon A background system program, started at boot, that runs without a user interface (a “service” on Windows).

Orphan A process whose parent has terminated; on Linux it is adopted by PID 1 or a subreaper.

PCB Process control block: the kernel data structure that represents one process.

PID Process identifier: the unique number of a process.

Process A program in execution.

Ready queue The list of processes that are ready to run and waiting for a CPU.

Shell A command interpreter; a system program that reads commands and runs them.

System call A controlled request from a program to the kernel, entered with a trap instruction.

System-call table The kernel’s array of handler addresses, indexed by system-call number.

System program A user-mode program shipped with the OS (file manager, compiler, `ls`, `ps`).

Wrapper A small library function that loads the registers and executes the trap for one system call.

Zombie A terminated process whose exit status has not yet been collected by its parent.

10 Try It Yourself

Look at processes on your own machine

Linux (lab PCs, WSL, or a virtual machine):

```
$ ps -ef | head            # all processes, with their PPID
$ ps -o pid,ppid,stat,cmd  # your processes and their states
$ pstree -p | less         # the process tree
$ cat /proc/$$/status      # the PCB of your own shell
$ strace -c ls             # system calls of ls, counted by type
$ make && ./fork_count 4   # in the Week 3 code folder
```

Windows: run `tasklist` in a Command Prompt; in Task Manager open the *Details* tab and show the PID column.

1. What is the PID of your shell, and which process is its parent?
2. How many processes are in state R right now? Why are there so few?
3. How many voluntary context switches has your shell made so far? Run a few commands and look again.

11 Review Questions

Conceptual

1. Explain the difference between an API function, a wrapper and a system call, using `printf()` and `write()` as examples. Does `strlen()` make a system call?
2. Describe the three methods of passing parameters to the OS. Which does Linux on x86-64 use, and why does it need two of them?
3. Classify each system call into one of the six categories: `fork()`, `chmod()`, `ioctl()`, `getpid()`, `socket()`, `close()`.
4. In which memory region does each of these live: a global `int count;`, a global `int max = 100;`, a local variable of `main()`, the memory returned by `malloc()`, the code of `main()`?
5. Draw the five-state diagram. For each transition, name an event that causes it. Why is there no arrow from waiting to running?
6. What is the difference between `fork()` and `exec()`? What does `fork()` return, and to whom?
7. What is a zombie process? What is an orphan? How does Linux make sure orphans do not remain zombies?

Numerical

8. A context switch takes $10\ \mu$s. What percentage of CPU time is overhead if the time slice is (a) 1 ms, (b) $50\ \mu$s?
9. A system performs 2000 context switches per second, each costing $4\ \mu$s. What fraction of the CPU is

lost? How many CPU-seconds is that per hour?

10. How many processes exist after this code runs, and how many times is `"hi"` printed?
```
fork(); fork(); fork(); fork(); printf("hi\n");
```

11. Process A computes for 2 ms, waits for the disk for 6 ms, then computes for 2 ms. Process B computes for 4 ms. Both are ready at $t = 0$, A runs first, there is one CPU and no time slice limit. Compute the total time and the CPU utilisation (a) if B starts only after A has finished, (b) if B runs while A waits.

12. On x86-64 Linux, a program calls `write(2, "Error\n", 6)`. What values are in `rax`, `rdi` and `rdx` just before the `syscall` instruction, and what is in `rax` afterwards if all bytes are written?

Answers to the numerical questions

8. (a) $10/(1000 + 10) \approx 0.0099 = 0.99\%$, about 1%. (b) $10/(50 + 10) \approx 0.167 = 16.7\%$: one sixth of the CPU is wasted.

9. $2000\ \text{s}^{-1} \times 4 \times 10^{-6}\ \text{s} = 8 \times 10^{-3} = 0.8\%$. Per hour: $0.008 \times 3600\ \text{s} = 28.8$ CPU-seconds.

10. Four forks: $2^4 = 16$ processes (15 created), and `"hi"` is printed 16 times.

11. (a) A takes $2 + 6 + 2 = 10$ ms, then B 4 ms: total 14 ms; the CPU is busy $2 + 2 + 4 = 8$ ms, so utilisation is $8/14 \approx 57\%$. (b) A runs 0–2, B runs 2–6, the CPU is idle 6–8, A runs 8–10: total 10 ms, utilisation $8/10 = 80\%$.

12. Before: `rax` $= 1$ (`write`), `rdi` $= 2$ (standard error), `rsi` $=$ the address of the string, `rdx` $= 6$. Afterwards: `rax` $= 6$, the number of bytes written.

12 Looking Ahead: Week 4

- Threads: one process, several flows of execution sharing the same memory.
- User threads and kernel threads; multithreading models (many-to-one, one-to-one, many-to-many).
- CPU scheduling basics: the CPU–I/O burst cycle, the short-term scheduler and the dispatcher.
- Scheduling criteria: CPU utilisation, throughput, turnaround time, waiting time and response time.

Further reading (optional)

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley: Chapter 2 (Sections 2.1–2.5, services, user interfaces, system calls, system programs) and Chapter 3 (Sections 3.1–3.3, process concept, scheduling queues and context switch, operations on processes).
- R. H. and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*:
  - Chapter 4, “The Abstraction: The Process”: `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-intro.pdf`
  - Chapter 5, “Interlude: Process API”: `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-api.pdf`
  - Chapter 6, “Mechanism: Limited Direct Execution”: `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-mechanisms.pdf`